

COMP9312 — Assignment 2 Solutions

YUHUI LUO (z5565446)

Q1. Graph homomorphism and induced subgraph matching

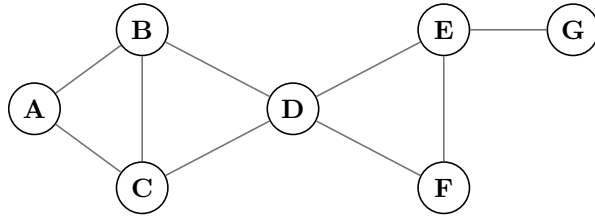


Figure 1: data graph G

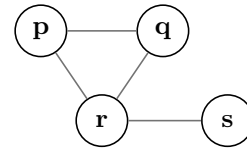


Figure 2: query graph P

(a) A homomorphism from P to G

A homomorphism only has to map every *edge* of P to an edge of G ; distinct vertices of P may collapse onto the same vertex of G . One exists — and here nothing even needs to collapse:

$$p \mapsto B, \quad q \mapsto C, \quad r \mapsto D, \quad s \mapsto E.$$

Edge of P	$p-q$	$p-r$	$q-r$	$r-s$
Image in G	$B-C$ ✓	$B-D$ ✓	$C-D$ ✓	$D-E$ ✓

Answer. A homomorphism from P to G exists, e.g. $p \mapsto B, q \mapsto C, r \mapsto D, s \mapsto E$ (all four edges of P land on edges of G).

(b) All induced subgraphs of G isomorphic to P

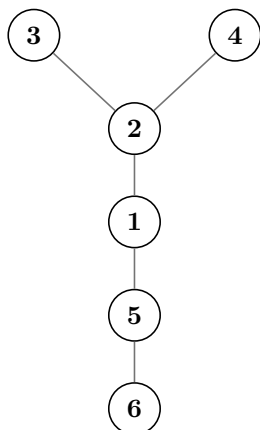
P is a triangle plus one pendant vertex (4 vertices, 4 edges). A vertex set $S \subseteq V(G)$ with $|S| = 4$ induces a copy of P **iff** three of its vertices form a triangle and the fourth sends *exactly one* edge into that triangle. An induced subgraph keeps *every* edge of G inside S , so two or three such edges would give 5 or 6 edges rather than 4, while none would leave the fourth vertex isolated and P is connected. Conversely any induced copy of P must contain a triangle, since P does, and its remaining vertex is P 's degree-1 pendant — so enumerating G 's triangles is exhaustive.

G has exactly three triangles: $\{A, B, C\}$, $\{B, C, D\}$, $\{D, E, F\}$. Extending each by a fourth vertex:

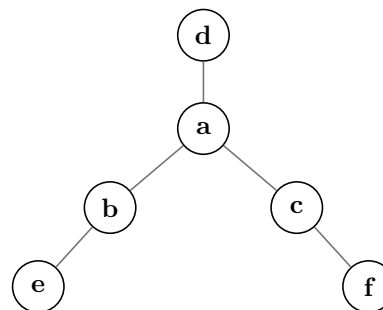
Triangle	4th vertex	Edges into the triangle	Induced copy of P ?
$\{A, B, C\}$	D	$D-B, D-C$ (2)	no
$\{A, B, C\}$	E	— (0)	no
$\{A, B, C\}$	F	— (0)	no
$\{A, B, C\}$	G	— (0)	no
$\{B, C, D\}$	A	$A-B, A-C$ (2)	no
$\{B, C, D\}$	E	$E-D$ (1)	yes
$\{B, C, D\}$	F	$F-D$ (1)	yes
$\{B, C, D\}$	G	— (0)	no
$\{D, E, F\}$	A	— (0)	no
$\{D, E, F\}$	B	$B-D$ (1)	yes
$\{D, E, F\}$	C	$C-D$ (1)	yes
$\{D, E, F\}$	G	$G-E$ (1)	yes

Answer. Exactly five vertex sets induce a subgraph isomorphic to P : $\{B, C, D, E\}$, $\{B, C, D, F\}$, $\{B, D, E, F\}$, $\{C, D, E, F\}$, $\{D, E, F, G\}$.

Q2. Weisfeiler-Lehman color refinement



G : a tree on 6 vertices



G' : a tree on 6 vertices

Both are trees on 6 vertices with the same degree sequence $[3, 2, 2, 1, 1, 1]$, so degrees alone cannot tell them apart.

Convention. A vertex's tuple is (own colour, sorted multiset of neighbour colours). Each round, the distinct tuples occurring in *either* graph are sorted lexicographically and receive the IDs $0, 1, 2, \dots$. Restarting the IDs at 0 in every round is what “starting from 0” prescribes. Continuing the numbering across rounds instead would only shift each round's IDs by a constant, padding the vectors with zero entries for the colours of earlier rounds; since $\phi_t(G)$ and $\phi_t(G')$ are always indexed the same way, every inner product — and hence the kernel — is unchanged either way.

Round 0

Every vertex has colour 0, so there is one colour and $\phi_0(G) = \phi_0(G') = [6]$.

Round 1

Lexicographic order of the flattened tuples: $(0, 0) < (0, 0, 0) < (0, 0, 0, 0)$, giving

ID	Tuple	Meaning
0	$(0, [0])$	degree-1 vertices
1	$(0, [0, 0])$	degree-2 vertices
2	$(0, [0, 0, 0])$	degree-3 vertices

G	Tuple	New colour	G'	Tuple	New colour
1	$(0, [0, 0])$	1	a	$(0, [0, 0, 0])$	2
2	$(0, [0, 0, 0])$	2	b	$(0, [0, 0])$	1
3	$(0, [0])$	0	c	$(0, [0, 0])$	1
4	$(0, [0])$	0	d	$(0, [0])$	0
5	$(0, [0, 0])$	1	e	$(0, [0])$	0
6	$(0, [0])$	0	f	$(0, [0])$	0

$$\phi_1(G) = [3, 2, 1], \quad \phi_1(G') = [3, 2, 1]$$

still identical — round 1 only counts degrees.

Round 2

Tuples are built from the round-1 colours above.

G	Colour	Neighbours' colours	Tuple	New colour
1	1	2(2), 5(1)	$(1, [1, 2])$	4
2	2	1(1), 3(0), 4(0)	$(2, [0, 0, 1])$	5
3	0	2(2)	$(0, [2])$	1
4	0	2(2)	$(0, [2])$	1
5	1	1(1), 6(0)	$(1, [0, 1])$	2
6	0	5(1)	$(0, [1])$	0

G'	Colour	Neighbours' colours	Tuple	New colour
a	2	$b(1), c(1), d(0)$	$(2, [0, 1, 1])$	6
b	1	$a(2), e(0)$	$(1, [0, 2])$	3
c	1	$a(2), f(0)$	$(1, [0, 2])$	3
d	0	$a(2)$	$(0, [2])$	1
e	0	$b(1)$	$(0, [1])$	0
f	0	$c(1)$	$(0, [1])$	0

Seven distinct tuples occur across the two graphs; sorted lexicographically they receive the IDs 0–6:

ID	0	1	2	3	4	5	6
Tuple	(0, [1])	(0, [2])	(1, [0, 1])	(1, [0, 2])	(1, [1, 2])	(2, [0, 0, 1])	(2, [0, 1, 1])
Count in G	1	2	1	0	1	1	0
Count in G'	2	1	0	2	0	0	1

$$\phi_2(G) = [1, 2, 1, 0, 1, 1, 0], \quad \phi_2(G') = [2, 1, 0, 2, 0, 0, 1]$$

Round 2 is the first round that separates the two trees: in G the degree-3 vertex sees two leaves and one degree-2 vertex, whereas in G' it sees one leaf and two degree-2 vertices.

(b) The two-round WL kernel

t	$\phi_t(G)$	$\phi_t(G')$	$\phi_t(G)^\top \phi_t(G')$
0	[6]	[6]	$6 \cdot 6 = 36$
1	[3, 2, 1]	[3, 2, 1]	$9 + 4 + 1 = 14$
2	[1, 2, 1, 0, 1, 1, 0]	[2, 1, 0, 2, 0, 0, 1]	$1 \cdot 2 + 2 \cdot 1 = 4$

Answer. $K_{\text{WL}}(G, G') = 36 + 14 + 4 = \mathbf{54}$.

Q3. Reachability indexing

(a) Minimal 2-hop labelling

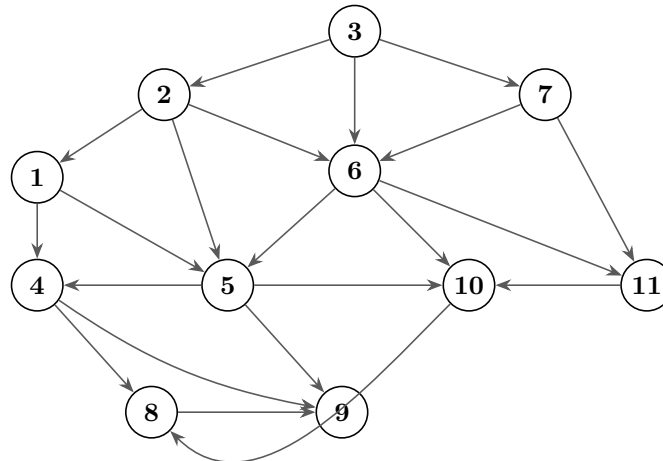


Figure 4: a DAG for minimal 2-hop labelling

Convention. $L_{\text{out}}(u)$ holds hubs reachable *from* u , $L_{\text{in}}(v)$ holds hubs that *reach* v , and

$$u \rightsquigarrow v \iff L_{\text{out}}(u) \cap L_{\text{in}}(v) \neq \emptyset.$$

Every vertex ends up in its own two labels. This is not a convention chosen by hand — it follows from the algorithm below. Each of hub k 's two searches starts *at* k , and the pruning test there is whether $L_{\text{out}}(k) \cap L_{\text{in}}(k)$ is already non-empty. In a DAG that intersection is necessarily empty before k labels itself: a common hub $h \neq k$ would give $k \rightsquigarrow h$ and $h \rightsquigarrow k$, i.e. a cycle. So k is never pruned at itself and always adds $k \in L_{\text{out}}(k)$, $k \in L_{\text{in}}(k)$. This is also exactly what makes the reflexive query $v \rightsquigarrow v$ answerable, since v is the only possible hub for it.

Processing order. Total degree = in-degree + out-degree, decreasing, ties broken by smaller vertex ID:

v	1	2	3	4	5	6	7	8	9	10	11
in-degree	1	1	0	2	3	3	1	2	3	3	2
out-degree	2	3	3	2	3	3	2	1	0	1	1
total	3	4	3	4	6	6	3	3	3	4	3

Order: 5, 6, 2, 4, 10, 1, 3, 7, 8, 9, 11

Algorithm. Take the hubs k in that order. For each k run a *backward* BFS (over reversed edges) and a *forward* BFS. On reaching a vertex x , first test whether the labels built *so far* already prove the corresponding reachability — $L_{\text{out}}(x) \cap L_{\text{in}}(k) \neq \emptyset$ backward, $L_{\text{out}}(k) \cap L_{\text{in}}(x) \neq \emptyset$ forward. If they do, **prune**: add nothing and do not expand past x , since everything behind x is already covered through that same earlier hub. Otherwise add k to $L_{\text{out}}(x)$ (backward) or $L_{\text{in}}(x)$ (forward) and keep expanding. The result is a *minimal* 2-hop cover: no single entry can be deleted without breaking some reachable pair.

Round-by-round trace (+ v = hub added at v , $\times v$ = pruned at v):

Hub k	Backward BFS $\rightarrow L_{\text{out}}$	Forward BFS $\rightarrow L_{\text{in}}$
5	+1, +2, +3, +5, +6, +7	+4, +5, +8, +9, +10
6	+2, +3, +6, +7	+6, +11; $\times 5, \times 10$
2	+2, +3	+1, +2; $\times 4, \times 5, \times 6$
4	+4; $\times 1, \times 5$	+4, +8, +9
10	+10, +11; $\times 5, \times 6, \times 7$	+8, +9, +10
1	+1; $\times 2$	+1; $\times 4, \times 5$
3	+3	+3, +7; $\times 2, \times 6, \times 11$
7	+7; $\times 3$	+7; $\times 6, \times 11$
8	+8; $\times 4, \times 10$	+8, +9
9	+9; $\times 4, \times 5, \times 8$	+9
11	+11; $\times 6, \times 7$	+11; $\times 10$

Two examples of the pruning. At hub 6 the forward BFS stops at 5 because $5 \in L_{\text{out}}(6)$ and $5 \in L_{\text{in}}(5)$ already certify $6 \rightsquigarrow 5$; everything behind 5 (namely 4, 8, 9, 10) is likewise covered through hub 5, so the search need not go on. At hub 8 the backward BFS stops at 4 and at 10, whose labels $L_{\text{out}}(4) = \{4\}$ and $L_{\text{out}}(10) = \{10\}$ already meet $L_{\text{in}}(8) = \{4, 5, 10\}$.

Final labels.

v	$L_{\text{in}}(v)$	$L_{\text{out}}(v)$
1	$\{1, 2\}$	$\{1, 5\}$
2	$\{2\}$	$\{2, 5, 6\}$
3	$\{3\}$	$\{2, 3, 5, 6\}$
4	$\{4, 5\}$	$\{4\}$
5	$\{5\}$	$\{5\}$
6	$\{6\}$	$\{5, 6\}$
7	$\{3, 7\}$	$\{5, 6, 7\}$
8	$\{4, 5, 8, 10\}$	$\{8\}$
9	$\{4, 5, 8, 9, 10\}$	$\{9\}$
10	$\{5, 10\}$	$\{10\}$
11	$\{6, 11\}$	$\{10, 11\}$
total	23 entries	21 entries

Sanity checks: $3 \rightsquigarrow 9$ because $L_{\text{out}}(3) \cap L_{\text{in}}(9) = \{5\}$ (indeed $3 \rightarrow 2 \rightarrow 5 \rightarrow 9$); $1 \not\rightsquigarrow 11$ because $L_{\text{out}}(1) \cap L_{\text{in}}(11) = \{1, 5\} \cap \{6, 11\} = \emptyset$, and everything reachable from 1 does stay inside $\{4, 5, 8, 9, 10\}$.

Answer. The minimal 2-hop labels are the 44 entries tabulated above (23 in L_{in} , 21 in L_{out}), obtained by pruned BFS with the hub order 5, 6, 2, 4, 10, 1, 3, 7, 8, 9, 11.

(b) Comparing two spanning trees

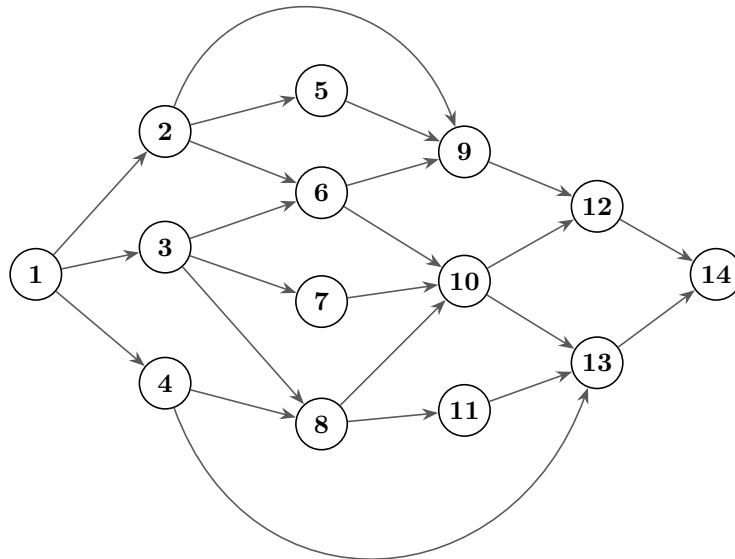


Figure 5: the DAG G (14 vertices, 23 edges)

Algorithm. Number the tree vertices by a **left-to-right post-order** traversal. Each vertex v receives the interval

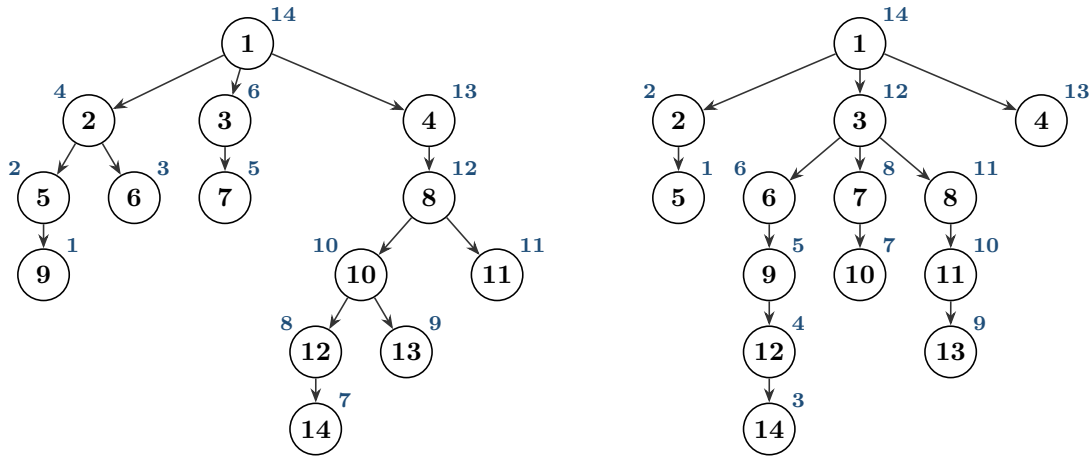
$$I(v) = [\min\{\text{post-numbers in } v\text{'s subtree}\}, \text{post}(v)],$$

which covers exactly v 's tree descendants. Then, in **reverse topological order**,

$$L(u) = I(u) \cup \bigcup_{u \rightarrow w \in E} L(w),$$

deleting from $L(u)$ any interval *contained* in another interval of the same label. Only containment is removed: adjacent or partially overlapping intervals are kept, since merging them would manufacture reachability that does not exist. A query is then $u \rightsquigarrow v \iff \text{post}(v)$ lies in some interval of $L(u)$.

Both spanning trees are rooted at vertex 1, the only source of the DAG. The post-order number of each vertex is printed in blue beside it.



Tree 1

Tree 2

Tree 1. Post-order: 9, 5, 6, 2, 7, 3, 14, 12, 13, 10, 11, 8, 4, 1.

v	post	$I(v)$	Final $L(v)$	$ L(v) $
1	14	$[1, 14]$	$[1, 14]$	1
2	4	$[1, 4]$	$[1, 4], [7, 10]$	2
3	6	$[5, 6]$	$[1, 1], [3, 3], [5, 6], [7, 12]$	4
4	13	$[7, 13]$	$[7, 13]$	1
5	2	$[1, 2]$	$[1, 2], [7, 8]$	2
6	3	$[3, 3]$	$[1, 1], [3, 3], [7, 10]$	3
7	5	$[5, 5]$	$[5, 5], [7, 10]$	2
8	12	$[7, 12]$	$[7, 12]$	1
9	1	$[1, 1]$	$[1, 1], [7, 8]$	2
10	10	$[7, 10]$	$[7, 10]$	1
11	11	$[11, 11]$	$[7, 7], [9, 9], [11, 11]$	3
12	8	$[7, 8]$	$[7, 8]$	1
13	9	$[9, 9]$	$[7, 7], [9, 9]$	2
14	7	$[7, 7]$	$[7, 7]$	1
Total intervals				26

Tree 2. Post-order: 5, 2, 14, 12, 9, 6, 10, 7, 13, 11, 8, 3, 4, 1.

v	post	$I(v)$	Final $L(v)$	$ L(v) $
1	14	[1, 14]	[1, 14]	1
2	2	[1, 2]	[1, 2], [3, 6], [7, 7], [9, 9]	4
3	12	[3, 12]	[3, 12]	1
4	13	[13, 13]	[3, 4], [7, 7], [9, 11], [13, 13]	4
5	1	[1, 1]	[1, 1], [3, 5]	2
6	6	[3, 6]	[3, 6], [7, 7], [9, 9]	3
7	8	[7, 8]	[3, 4], [7, 8], [9, 9]	3
8	11	[9, 11]	[3, 4], [7, 7], [9, 11]	3
9	5	[3, 5]	[3, 5]	1
10	7	[7, 7]	[3, 4], [7, 7], [9, 9]	3
11	10	[9, 10]	[3, 3], [9, 10]	2
12	4	[3, 4]	[3, 4]	1
13	9	[9, 9]	[3, 3], [9, 9]	2
14	3	[3, 3]	[3, 3]	1
Total intervals				31

Which tree gives the better labelling?

	Tree 1	Tree 2
Total intervals (index size)	26	31
Largest single label	4 (vertex 3)	4 (vertices 2, 4)
Vertices with a single interval	6 (1, 4, 8, 10, 12, 14)	5 (1, 3, 9, 12, 14)
Label-size distribution (1/2/3/4)	6/5/2/1	5/3/4/2
Non-tree edges absorbed at no cost	2 (2 $\rightarrow$ 9, 4 $\rightarrow$ 13)	0

Why. A vertex's label starts as its own interval and then grows by inheriting intervals along its out-edges. Two distinct effects limit that growth.

- A non-tree edge $u \rightarrow w$ is *free* whenever w is already a tree descendant of u , because the tree path from u down to w carries $L(w)$ upward in any case. Note this is *not* a containment effect: in Tree 1, $L(9) = \{[1, 1], [7, 8]\}$ is not inside $I(2) = [1, 4]$, yet the edge $2 \rightarrow 9$ still costs nothing.
- Containment is the stronger effect, and the one that decides this comparison. If everything u can reach lies inside u 's own subtree, then every inherited interval has both endpoints among that subtree's post-numbers, so it is contained in $I(u)$ and deleted — and $L(u)$ collapses to the single interval $I(u)$. In both trees $L(u) = \{I(u)\}$ holds *exactly* for the vertices counted in the “single interval” row above.

So a spanning tree is good precisely when it keeps mutually reachable clusters together inside one subtree.

- **Tree 1** puts the DAG's whole converging tail — 4, 8, 10, 11, 12, 13, 14 — into a single subtree rooted at 4 ($4 \rightarrow 8 \rightarrow \{10, 11\}$, $10 \rightarrow \{12, 13\}$, $12 \rightarrow 14$). Every DAG edge among those vertices therefore stays inside that subtree, so $L(4)$ collapses to the one interval $[7, 13]$, and likewise $L(8) = [7, 12]$, $L(10) = [7, 10]$, $L(12) = [7, 8]$. On top of that, the non-tree edges $2 \rightarrow 9$ and $4 \rightarrow 13$ point back into the subtree they start from and cost nothing at all.
- **Tree 2** splits that same tail across three sibling branches of vertex 3 ($6 \rightarrow 9 \rightarrow 12 \rightarrow 14$, $7 \rightarrow 10$, $8 \rightarrow 11 \rightarrow 13$) and leaves vertex 4 as a *leaf*. A leaf's interval $I(4) = [13, 13]$ is a single point that can absorb nothing, so $4 \rightarrow 8$ and $4 \rightarrow 13$ push $L(4)$ up to four intervals. The cross-branch edges $6 \rightarrow 10$, $8 \rightarrow 10$, $10 \rightarrow 12$, $10 \rightarrow 13$ and $13 \rightarrow 14$ are likewise all non-tree edges between *different* branches, so their intervals propagate upward instead of being swallowed. That is why six of Tree 2's vertices need three or four intervals, against three in Tree 1.

Note that the *maximum* label size is 4 for both trees, so what separates them is the total index size — which is also what determines the space used and the cost of a query, since answering $u \rightsquigarrow v$ scans $L(u)$.

Answer. **Tree 1** gives the better tree-cover labelling: 26 intervals against Tree 2's 31 (16% smaller), because Tree 1 keeps the DAG's converging tail $\{4, 8, 10, 11, 12, 13, 14\}$ inside one subtree, whereas Tree 2 scatters it over three sibling branches and leaves vertex 4 as an absorbing-nothing leaf.

Q4. One-layer GNN mean aggregation

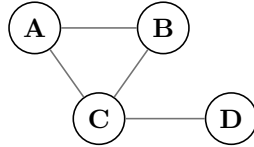


Figure 7: graph for Q4

$$h_A^{(0)} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad h_B^{(0)} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad h_C^{(0)} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad h_D^{(0)} = \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \quad W = \begin{pmatrix} 1 & -1 & 1 & 0 \\ 0 & 1 & -1 & 1 \end{pmatrix}$$

(a) Vertices B and C

Vertex B , with $N(B) = \{A, C\}$:

$$h_{N(B)}^{(0)} = \frac{1}{2} \left[\begin{pmatrix} 1 \\ 0 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} \right] = \begin{pmatrix} 1 \\ 0.5 \end{pmatrix}, \quad \text{CONCAT}(h_B^{(0)}, h_{N(B)}^{(0)}) = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0.5 \end{pmatrix}$$

$$W \cdot \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0.5 \end{pmatrix} = \begin{pmatrix} 1(0) - 1(1) + 1(1) + 0(0.5) \\ 0(0) + 1(1) - 1(1) + 1(0.5) \end{pmatrix} = \begin{pmatrix} 0 \\ 0.5 \end{pmatrix} \xrightarrow{\text{ReLU}} h_B^{(1)} = \begin{pmatrix} 0.00 \\ 0.50 \end{pmatrix}$$

Vertex C , with $N(C) = \{A, B, D\}$:

$$h_{N(C)}^{(0)} = \frac{1}{3} \left[\begin{pmatrix} 1 \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} + \begin{pmatrix} 2 \\ 0 \end{pmatrix} \right] = \begin{pmatrix} 1 \\ 1/3 \end{pmatrix}, \quad \text{CONCAT}(h_C^{(0)}, h_{N(C)}^{(0)}) = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1/3 \end{pmatrix}$$

$$W \cdot \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1/3 \end{pmatrix} = \begin{pmatrix} 1 - 1 + 1 + 0 \\ 0 + 1 - 1 + \frac{1}{3} \end{pmatrix} = \begin{pmatrix} 1 \\ 1/3 \end{pmatrix} \xrightarrow{\text{ReLU}} h_C^{(1)} = \begin{pmatrix} 1.00 \\ 0.33 \end{pmatrix}$$

v	$N(v)$	Neighbour mean	Concatenated input	$h_v^{(1)}$
B	$\{A, C\}$	$(1, 0.5)^\top$	$(0, 1, 1, 0.5)^\top$	$(\mathbf{0.00}, \mathbf{0.50})^\top$
C	$\{A, B, D\}$	$(1, 1/3)^\top$	$(1, 1, 1, 1/3)^\top$	$(\mathbf{1.00}, \mathbf{0.33})^\top$

Answer. $h_B^{(1)} = (0.00, 0.50)^\top$ and $h_C^{(1)} = (1.00, 0.33)^\top$.

(b) After adding the edge $B-D$

Only $N(B)$ and $N(D)$ change, and no initial embedding changes, so A and C receive exactly the same input vector as before. Since ReLU can also hide a change by clipping it to 0, all four vertices are computed explicitly:

v	$N(v)$ before $\rightarrow$ after	$h_v^{(1)}$ before	$h_v^{(1)}$ after	Changed?
A	$\{B, C\} \rightarrow \{B, C\}$	$(1.50, 0.50)^\top$	$(1.50, 0.50)^\top$	no
B	$\{A, C\} \rightarrow \{A, C, D\}$	$(0.00, 0.50)^\top$	$(\mathbf{0.33}, \mathbf{0.00})^\top$	yes
C	$\{A, B, D\} \rightarrow \{A, B, D\}$	$(1.00, 0.33)^\top$	$(1.00, 0.33)^\top$	no
D	$\{C\} \rightarrow \{B, C\}$	$(3.00, 0.00)^\top$	$(\mathbf{2.50}, \mathbf{0.50})^\top$	yes

The two recomputations:

$$\begin{aligned}
 B : \quad h_{N(B)}^{(0)} &= \frac{1}{3} \left[\begin{pmatrix} 1 \\ 0 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} + \begin{pmatrix} 2 \\ 0 \end{pmatrix} \right] = \begin{pmatrix} 4/3 \\ 1/3 \end{pmatrix}, & W \cdot \begin{pmatrix} 0 \\ 1 \\ 4/3 \\ 1/3 \end{pmatrix} &= \begin{pmatrix} 0 - 1 + \frac{4}{3} + 0 \\ 0 + 1 - \frac{4}{3} + \frac{1}{3} \end{pmatrix} = \begin{pmatrix} 1/3 \\ 0 \end{pmatrix} \\
 D : \quad h_{N(D)}^{(0)} &= \frac{1}{2} \left[\begin{pmatrix} 0 \\ 1 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} \right] = \begin{pmatrix} 0.5 \\ 1 \end{pmatrix}, & W \cdot \begin{pmatrix} 2 \\ 0 \\ 0.5 \\ 1 \end{pmatrix} &= \begin{pmatrix} 2 - 0 + 0.5 + 0 \\ 0 + 0 - 0.5 + 1 \end{pmatrix} = \begin{pmatrix} 2.5 \\ 0.5 \end{pmatrix}
 \end{aligned}$$

Both are already non-negative, so ReLU leaves them unchanged: $h_B^{(1)} = (0.33, 0.00)^\top$ (was $(0.00, 0.50)^\top$) and $h_D^{(1)} = (2.50, 0.50)^\top$ (was $(3.00, 0.00)^\top$).

Answer. The affected vertices are $\{\mathbf{B}, \mathbf{D}\}$ — exactly the endpoints of the new edge. A and C are unchanged because a one-layer GNN only reads the 1-hop neighbourhood, and neither $N(A)$ nor $N(C)$ was touched by adding B – D .